Table of Contents

1. Introduction to time series

This chapter introduces the fundamental concepts and techniques of time series analysis, laying the groundwork for the analysis of forecasting methods, that will be explored in the following chapters. It begins by defining time series data and its unique temporal dependencies, before examining key analytical concepts such as stationarity and autocorrelation. Furthermore, the chapter explores the opportunities and challenges associated with time series forecasting.

1.1 Definition of time series

A time series is a set of observations X_t, each one being recorded at a specific time t. Time series can be either discrete or continuous:

- A discrete-time series is one in which the set T_0 of times at which observations are made is a discrete set, as is the case, for example, when observations are made at fixed time intervals.
- Continuous time series are obtained when observations are recorded continuously over some time interval, e.g., when $T_0 = [0, 1]$.

For the purpose of this study, the focus is mainly on discrete-time time series. Thus, a time series is defined as a sequence of observations recorded at specific time intervals, capturing data at equally spaced points in time, whether in years, days, minutes or any other time measure.

Time series data are unique because they measure one or more variables as they change over time, creating dependencies between data points. Unlike cross-sectional datasets, where observations are assumed to be independent, time series show serial correlation, meaning that each time point in a time series dataset is influenced by its predecessor. Consequently, past observations can have a significant impact on future observations and require specialized techniques to be analyzed.

1.2 Time series analysis and forecasting

Time series analysis is a discipline that systematically studies time series data, in order to identify and model the structures within data, including autocorrelation, seasonal patterns and trends. Time series forecasting is a key element of this analysis, which uses historical data to make future predictions.

Forecasting problems can be categorized into two main types:

- Univariate: problems where only one variable and its variation over time are involved, the objective of this analysis is to predict future values based only on previous observations.
- Multivariate: these problems involve multiple variables that influence the target variable, these variables provide more context and may improve the accuracy of predictions.

Furthermore, forecasting models can be classified based on the time horizon of their predictions:

- One-step models predict only the next point in a series. To create multi-step forecasts with these models, you can repeatedly use the previous prediction as input for the next step. However, this approach can extend any existing errors over multiple steps.
- Multi-step models are designed to predict multiple future points simultaneously. These models are generally better for long-term forecasts and perform well for single-step forecasts.

1.3 Applications and challenges

Time series forecasting is a critical tool for many companies, as it enables informed decision making and advanced predictive models, that allows companies to forecast sales, costs, demand and risks with great precision and speed.

This technique is relevant across various industries: in the consumer and retail sector, forecasting enables businesses to automate revenue predictions and workforce planning. It also enhances supply chain management by providing real-time insights into sales, costs, and raw material demand.

The technology industry benefits from time series forecasting through advanced IoT-based models that enhance sales predictions by up to 80%.

Financial institutions use time series models to forecast stock prices, asset prices and macroeconomic factors, enabling better investment strategies and risk assessments.

Additionally, economic forecasting is widely used by global organizations such as the World Trade Organization (WTO) to predict international trade trends, while the US Federal Reserve relies on time series models to set interest rates and manage monetary policy effectively. Other applications include e-commerce, where forecasting is used to predict sales trends, manage inventory and optimize pricing strategies. Furthermore, the global time series forecasting market was valued at approximately $290 million in 2023 and is projected to reach $450 million by 2032, indicating an increasing need for companies and organizations to leverage these techniques.

However, time series forecasting presents several challenges that can impact its effectiveness. One key limitation is its restricted applicability, as forecasting techniques are designed exclusively for time-dependent data. Another challenge lies in interpretation, particularly when transformations such as differencing or logarithmic scaling are applied. While these techniques enhance model performance, they can make it difficult to derive actionable insights. Similarly, the issue of generalization arises when models trained on specific datasets fail to adapt to new conditions, limiting their predictive accuracy in dynamic environments where external factors or structural changes alter underlying patterns.

The complexity of model selection further complicates time series forecasting, as the choice of an inappropriate model can lead to lower accuracy in predictions.

Finally, data availability remains a fundamental challenge, as effective forecasting often requires extensive historical data to capture trends and seasonality accurately. In many cases, such data is limited, constraining the model's ability to produce reliable predictions.

1.4 Components of time series data

Time series data typically consists of different components that define its behavior over time. Analyzing these components is crucial for a better understanding of temporal patterns. The main components are:

- Trend: represents the general long-term direction of data, indicating whether it follows an increasing, decreasing or stable trajectory over an extended period of time.
- Seasonality: refers to periodic behaviors and recurring patterns that occur at regular intervals (such as seasons of the year). Seasonal components exhibit fluctuations fixed in timing, direction, and magnitude.
- Cycles: demonstrate fluctuations that do not follow a fixed periodicity, such as economic expansions and recessions. Unlike seasonal variations, these patterns do not have consistent amplitudes or durations.
- Residuals: explain the random variability in the data that is not explained by the other components.

Decomposition methods are widely used in time series analysis to break down these components, in order to better understand the underlying structures of a time series.

The most basic decomposition techniques include Classical Decomposition, which assumes a fixed seasonal pattern, and Moving Average Smoothing, which extracts the trend by averaging out short-term fluctuations.

One of the most versatile and robust methods for decomposition is the Seasonal and Trend decomposition using Loess (STL) method, which uses logically weighted regression to decompose the time series. Unlike traditional methods, STL is widely used because it allows to control the degree of smoothing for trend and seasonality separately, making it more adaptable for real-world data.

Beyond decomposing time series data into its fundamental components, it is essential to examine its statistical properties to ensure meaningful analysis and forecasting. Two key concepts in this are stationarity and autocorrelation.

In general, a time series $\{X_t$, t=0,±1, . . .} is defined as stationary if it has statistical properties that are similar to those of "time-shifted" series $\{X_{(t+h)}$, t = 0,±1, . . .}, for each integer h.
To formally introduce the concept of stationarity we need to define the mean function and the covariance function:

- Let $\{X_t\}$ be a time series with E $(X_t)^2 < \infty$. The mean function of $\{X_t\}$ is: $\mu_x(t)$ = E (X_t).

Thus, we can now define (weakly) stationarity:

$\{X_t\}$ is (weakly) stationary if
(i) $\mu_x(t)$ is independent of t,
and
(ii) $\mu_x(t+h, t)$is independent of t for each h.

As a consequence, the statistical properties of stationary time series do not depend on the specific time at which the series is observed, in fact, time series with trends or seasonality are typically non-stationary, whereas a white noise series is stationary.

Autocorrelation measures the linear relationship between lagged values of a time series. The value of the autocorrelation coefficient can be written as:

$$r_k = \frac{\sum_{t=k+1}^{T}(y_t - \bar{y})(y_{t-k} - \bar{y})}{\sum_{t=1}^{T}(y_t - \bar{y})^2}$$

Where k represents the time lag i.e. the number of time steps separating the current value from the past value used in the calculation of autocorrelation.

The autocorrelation coefficients constitute the autocorrelation function (ACF). The autocorrelation may assume both positive and negative values:

- ACF > 0: the ACF tends to be positive when time series data follows a trend, in this situation, high values tend to be followed by other high values, and low values by low values.
- ACF<0: the ACF is negative when high values are likely to be followed by low values, and vice versa.

In time series analysis, ACF plots are commonly used to detect randomness and autocorrelation within a dataset. An ACF plot consists of bars at each lag, where the height of each bar represents the correlation coefficient at that lag. If the plot shows a gradual decline in bar heights, it suggests a long-term dependency on data; whereas regular spikes at certain lags suggest seasonality in the data.

2. Time Series Forecasting Models

The aim of this section is to present and compare various time series forecasting models, beginning with basic approaches like smoothing techniques and progressing to more advanced models such as ARIMA and SARIMAX. It also introduces novel, efficient models methods and describes powerful yet complex deep learning- based models.

2.1 Smoothing techniques

Smoothing techniques are methods used to reduce noise and short-term fluctuations improving the forecast of time series. The most often used smoothing approaches are the Moving Average and Exponential Smoothing (single, double, triple). The basic idea of smoothing models is to make predictions of future values as weighted averages of past observations. These techniques are usually used with univariate time series.

2.1.1 Equally Weighted Moving Average

Moving Average models determine the averages of observed values that surround a particular time.

In equally weighted moving average models, we compute an equally weighted average, meaning we give the same weight to all the observations in the considered interval.

Let Y be the variable of interest and t the time dimension, with moving averages techniques we estimate the value of Y at time $t+1$ as the average of the most recent m observations:

$$\widehat{Y_{t+1}} = \frac{1}{m}\sum_{i=0}^{m-1} Y_{t-i}$$

We observe that with m=1 the forecast for the next step will be the most recently observed value; thus, the simple moving average model is equivalent to a random walk model with m=1. In fact, with a random walk model, the current value of a time series will depend only on the previous value plus some random noise:

$$\widehat{Y_{t+1}} = Y_t + \epsilon_t$$

From this brief introduction, it is clear that the simple moving average provides a good estimate of the mean of time series if the mean is constant or slowly changing, in fact, simple moving average models are more suitable for stationary time series.

In general, this kind of technique is useful for extracting the key patterns within data and eliminating noise. However, due to the simplicity of the model and the stationarity assumption, equally weighted moving average techniques often do not perform well from a forecasting perspective: these methods lag behind trends, especially in the presence of strong and rapid trends, causing significant forecasting errors.

2.1.2 Exponential Smoothing

Exponential smoothing differs from equally weighted moving average techniques for the fact that the average is computed with exponential weights, assigning higher weights to more recent observations. Furthermore, moving average techniques completely ignore observations preceding those in the m-long window.

As we assign higher weights to recent observations, exponential smoothing will be more sensitive to local changes than the equally weighted moving average method. The main exponential smoothing methods are: Single, Double (Brown's), and Triple (Holt-Winters') Exponential Smoothing.

2.1.2.1 Single Exponential Smoothing

In single exponential smoothing, forecasts are computed with weighted averages, where the weights decrease exponentially. We denote α in [0,1] as the smoothing constant, that controls the rate at which weights decrease.

Let L be the current level of the series as estimated from the data up to the present, the value of L is computed as:

$$L_t = \alpha Y_t + (1-\alpha)L_{t-1}$$

and $\widehat{Y_{t+1}} = L_t$, as a result, the next forecast is a linear combination of the previous observation and the last forecast: $\widehat{Y_{t+1}} = \alpha Y_t + (1-\alpha)\widehat{Y_t}$.

The single exponential smoothing can be formulated as:

$$\widehat{Y_{t+1}} = \alpha[Y_t + (1-\alpha)Y_{t-1} + (1-\alpha)^2 Y_{t-2} + (1-\alpha)^3 Y_{t-3} + \cdots]$$

Even if single exponential smoothing is able to predict better local changes, it is not able to predict trend and seasonality.

2.1.2.2 Double Exponential Smoothing – Brown's

Double exponential smoothing has the ability to capture trend, this is done by adding a second component b, representing the trend. The formulation is:

$$S_t = \alpha Y_t + (1-\alpha)(S_{t-1} + b_{t-1})$$

$$b_t = \beta(S_t - S_{t-1}) + (1-\beta)b_{t-1}$$

The first equation updates the smoothed value of the series by combining the current observation with the previous smoothed value and the trend component. Meanwhile, the second equation updates the trend estimate by considering the change in the smoothed values and applying a weighted average with the previous trend.

In the second equation, similarly to α, β controls how much weight is given to recent trend changes compared to previous trends.

As a result, the future prediction is obtained as the sum of these two components:

$$\widehat{Y_{t+1}} = S_t + b_t$$

Even if double exponential smoothing incorporates trends, it is not able to incorporate seasonality.

2.1.2.3 Triple Exponential Smoothing – Holt's

Triple exponential smoothing extends double exponential smoothing by incorporating seasonality, similar to the double exponential smoothing, triple exponential smoothing incorporates seasonality by adding a third component c:

$$(i) \quad S_t = \alpha(Y_t - c_{t-L}) + (1-\alpha)(S_{t-1} + b_{t-1})$$

$$(ii) \quad b_t = \beta(S_t - S_{t-1}) + (1-\beta)b_{t-1}$$

$$(iii) \quad c_t = \gamma(Y_t - S_{t-1} - b_{t-1}) + (1-\gamma)c_{t-L}$$

The first equation updates the smoothed value of the series by adjusting for seasonal effects and combining the current observation with the previous smoothed value and the trend component.

The second equation, functions similarly to the double exponential smoothing case.

The third equation updates the seasonal component based on the deviation from the trend-adjusted level.

In these equations L represents the length of the seasonal cycle, and γ controls how much weight is given to recent seasonal changes compared to previous seasonality.

As a result, the prediction in m steps will be:

$$\widehat{Y_{t+m}} = (S_t + m \cdot b_t) \cdot c_{t-L+(m-1)} \ mod\ L$$

Finally, exponential smoothing techniques effectively forecast univariate time series with short-term trends and simple seasonal patterns, while remaining computationally efficient. However, these methods have limitations, particularly in handling time series with complex patterns or anomalies, such as sudden trend changes or external shocks. In these cases, more sophisticated forecasting techniques may be preferred. Additionally, the selection of parameters can affect the precision of the forecasts.

2.2 ARIMA Models

ARIMA models, alongside exponential smoothing models, are among the most widely used approaches to time series forecasting. While exponential smoothing relies on the concepts of trend and seasonality, ARIMA models leverage the autocorrelations within the data.

2.2.1 ARMA Model

The ARMA (Auto-Regressive Moving Average) model serves as the foundational building blocks for ARIMA models, as the latter extend ARMA processes by incorporating an additional differencing component to achieve stationarity in non-stationary time series, in fact, a fundamental assumption of ARMA models is that the time series is stationary.

ARMA models combine two models:

- The Auto-Regressive (AR) model, that anticipate series' dependence on its own past values.
- The Moving Average (MA) model, that anticipate series' dependence on past forecast errors.

In an Auto-Regressive model, the target variable is forecasted using a linear combination of past values of the variable. The term *autoregression* indicates that it is a regression of the variable against itself.

An AR(p) model (Auto-Regressive model of order p) can be written as:

$$y_t = c + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \cdots + \phi_p y_{t-p} + \varepsilon_t$$

This formulation resembles multiple regression but with lagged values, with $y_t$serving as a predictor. The parameter p specifies the number of past values of the time series used in the prediction.

In contrast, a Moving Average does not use past values of the target variable but rather past forecast errors to make predictions. With this kind of models, y_t can be thought of as a weighted moving average of the past few forecast errors.

An MA(q) model (Moving Average model of order q) can be written as:

$$y_t = c + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \theta_2 \varepsilon_{t-2} + \cdots + \theta_q \varepsilon_{t-q}$$

The parameter q, similarly to p, indicates the number of past error terms used to model the current value of the time series.

By combining autoregression with a moving average model, the ARMA(p,q) model is obtained. It can be written as:

$$y_t = c + \sum_{i=1}^{p} \phi_i y_{t-i} + \sum_{j=1}^{q} \theta_j \varepsilon_{t-j} + \varepsilon_t$$

The selection of the parameters p and q in ARMA models can be guided by the autocorrelation function plot for q and the partial autocorrelation function plot for p. Alternatively, they can be treated as hyperparameters and optimized through different built-in methods or cross-validation.

2.2.2 ARIMA Model

The ARIMA (Autoregressive Integrated Moving Average) model represents an evolution of the ARMA model, extending its applicability to non-stationary time series.

Differencing is a transformation used in ARIMA models to convert a non-stationary time series into a stationary one by computing the difference between consecutive observations, and it is introduced through the integration (I) component. This process,

known as first differencing, is expressed as $y'_t = y_t - y_{t-1}$ and helps remove linear trends or other forms of non-stationarity in the mean.

The full model ARIMA(p, d, q) can be written as:

$$y'_t = c + \sum_{i=1}^{p} \phi_i y'_{t-i} + \sum_{j=1}^{q} \theta_j \varepsilon_{t-j} + \varepsilon_t$$

Where y'_t is the differenced series and d is the degree of first differencing involved, meaning the number of times the data needs to be differenced to achieve stationarity. For example, if d = 1, the model uses first differencing; if d = 2, it applies first differencing twice, effectively computing the difference of the differences. In other words, d indicates the minimum number of differencing steps needed to make the time series stationary.

2.2.3 SARIMA Models

The SARIMA (Seasonal Autoregressive Integrated Model) is an extension of the non-seasonal ARIMA model, designed to handle data with seasonality.

The seasonal component (represented by the "S" in SARIMA), accounts for repeating patterns at regular intervals. Seasonality is incorporated into SARIMA through seasonal differencing, which – unlike standard differencing - subtracts the time series data by a lag that equals the seasonal period.

The equation for the SARIMA(p, d, q)(P,D,Q) can be written as:

$$y'_t = c + \sum_{i=1}^{p} \phi_i y'_{t-i} + \sum_{k=1}^{P} \Phi_k y'_{t-k} + \sum_{j=1}^{q} \theta_j \varepsilon_{t-j} + \sum_{m=1}^{Q} \Theta_m \varepsilon_{t-m} + \varepsilon_t$$

Where:

- the second and fourth term represents the Auto-Regressive and Moving Average components respectively
- the third term represents the Seasonal Auto-Regressive component, that captures the seasonal patterns by linking current values to past values at seasonal lags
- the fifth component captures the seasonal effects in the error structure.

Similarly to the non-seasonal ARIMA case, P represents how many past seasonal values influence the current value, Q indicates how many past seasonal forecast errors affect the current value and D is the seasonal differencing order. Finally, m represents the length of the seasonal cycle.

Finally, SARIMAX (Seasonal ARIMA with eXogenous variables), is an extension of the SARIMA model as it can be used for multivariate forecasting problems, as it effectively incorporates exogenous variables into the analysis to improve prediction accuracy. This model introduces two major elements: covariates, the external variables that

influence the time series, and the covariate component that models the effect of covariates on the time series.

In conclusion, the ARIMA family of models are powerful tools for time series forecasting, excelling in capturing linear dependencies, trends, and seasonality, making them widely applicable across various domains. However, they struggle with complex patterns that change over time, such as sudden shifts or long-term trends that aren't easily captured by past values. They also have trouble handling very irregular or highly non-linear data, where relationships between points are not simple. Additionally, these models require careful tuning and pre-processing, which can make them harder to use for big or complex datasets.

2.3 Prophet

Prophet is a forecasting method developed by Facebook (by Sean J. Taylor and Ben Letham), and released in 2017 as an open source software. The model was designed to overcome common issues associated with traditional automated forecasting methodologies. Automated forecasting tools (such as ARIMA) are systems that generate forecasts automatically in a reliable way, making them accessible even to non-experts while ensuring fast results. However, these methods often rely on fixed assumptions, resulting in limited flexibility, and parameter tuning can be complex and prone to significant errors. On the other hand, more complex and accurate forecasting tools require more experience and highly skilled analysts who are often unavailable in business contexts.

Taylor and Letham developed a flexible automated forecasting tool that is both fast and accessible to non-experts, making it ideal for business applications. Prophet is specifically designed to handle business time series data, which often exhibit the following typical characteristics that the model is programmed to manage:

- Time series data captured at hourly, daily, weekly or monthly level
- Seasonality effects (daily, weekly or yearly) which Prophet automatically detects
- Holidays and other special events that do not necessarily follow the seasonal patterns
- Missing data and outliers
- Important trend changes (captured by Prophet's trend models)

Furthermore, Prophet is able to provide uncertainty estimates for forecasted values, and parameters tuning is relatively intuitive, not requiring detailed understanding of the model's internal mechanisms.

2.3.1 The Prophet model

Prophet is designed as an additive model:

$$y(t) = g(t) + s(t) + h(t) + \epsilon_t$$

Where:

- $g(t)$ describes a piecewise linear or logistic trend
- $s(t)$ describes the seasonal patterns
- $h(t)$ describes the holiday effects
- ϵ_t is a white noise error term

Trend Model

Two trend models are implemented to cover many of Facebook's use cases: a saturating growth model and a piecewise linear model.

- The saturating growth model is commonly used to model population growth (similarly as Facebook's users growth) where there is a non-linear growth until the capacity is saturated. This behavior is modeled with the logistic growth model:

$$g(t) = \frac{C}{1 + \exp\big(-k(t - m)\big)}$$

Where C represents the carrying capacity, k the growth rate and m a shift parameter. Prophet extends the classic logistic model by introducing a time-varying capacity $C(t)$, and allowing the growth rate to vary.
To model variabilities in trend, Prophet introduces changepoints where the growth rate can change. Suppose there are S changepoints at different times, thus the growth rate at time t is the base rate plus the sum of rate change δ at each changepoint:

$$k_t = k + a(t)^T \delta$$

Where:

$$a_j(t) = 1 \text{ if } t \geq s_j, \text{ otherwise } 0$$

Then the offset parameter is adjusted accordingly. The piecewise logistic growth model is thus expressed as:

$$g(t) = \frac{C(t)}{1 + \exp\Big(-(k + a(t)^T \delta)\big(t - (m + a(t)^T \gamma)\big)\Big)}$$

- The linear model with changepoints is used when problems do not have a saturating growth, the model is formulated as:

$$g(t) = (k + a(t)^T \delta)t + (m + a(t)^T \gamma)$$

Where k is the growth rate, δ is the rate of adjustment and m is the offset parameter.

Changepoints can be either specified by analysts or selected automatically. Prophet uses a Bayesian approach to perform automatic selection by placing a prior on δ, then the number of changepoints is specified by the analysts and the prior used is: $\delta_j \sim$

Laplace$(0, \tau)$. Where τ controls the flexibility of the model. Furthermore, Prophet is also able to deliver the uncertainty of the trend forecast.

Seasonality model

To model periodic effects, a flexible Fourier series approach is used, where the period is set based on the time scale of the data (for example, it can be yearly or weekly). This method approximates smooth seasonal effects by estimating a set of parameters that represent the seasonal patterns. By adjusting the number of terms in the series, the model can capture either slow or fast-changing seasonal patterns, balancing flexibility and the risk of overfitting.

Lastly, the holiday effects are added as simple dummy variables.

2.3.2 Final considerations on Prophet

Prophet has gained popularity due to its ability to produce accurate and interpretable forecasts while remaining easy to use. It has applications in various domains, including retail, finance, demand forecasting, and web analytics. One of its key strengths is its flexibility and, unlike traditional ARIMA models, Prophet does not require measurements to be regularly spaced and can handle missing values. The model fitting is fast, enabling analysts to interactively explore different model specifications. Given its regression-based structure, it is also easy for analysts to extend the model by incorporating new components when necessary. However, while Prophet performs well in many scenarios, it may struggle to capture complex relationships or irregular patterns that require more advanced modeling techniques.

2.4 Neural Networks and Deep Learning for time series

Neural networks, particularly deep learning networks, have recently gained popularity in time series forecasting, as they are able to model complex patterns within data. Deep learning is a subfield of machine learning that leverages artificial neural networks to learn from data. It is widely used for a variety of purposes, including image recognition, natural language processing and sequence modelling tasks.

Deep learning models can autonomously learn and extract complex structures and essential characteristics in time series data, such as trends, seasonality and autocorrelation. Notably, these techniques do not require prior specification of trend components or seasonal frequencies to model time series effectively. Furthermore, this class of models is capable of capturing many types of correlations and multiple seasonality: for example, they are able to capture both the yearly and monthly cycles. Additionally, they can jointly model related time series, for instance, when similar patterns exist across different series, the model is able to learn and exploit these relationships.

However, the complexity of deep learning models may introduce several challenges. Firstly, these models are generally less transparent and more complex to implement compared to traditional approaches, and they are also more computationally expensive. In some cases, the results can be difficult to interpret, as many of these

models are “black-box” models. Finally, the complexity of the model may lead to overfitting if not properly regularized.

Time series modelling falls within sequence modelling tasks, and Recurrent Neural Networks (RNNs) are among the most commonly used architectures for this purpose.

2.4.1 Recurrent Neural Networks (RNN)

Recurrent Neural Networks were first introduced by Elman. These networks are able to map a sequence of inputs (i.e., historical steps of a time series) to a predicted output (i.e., the next step in the series).

The core idea behind RNN is the use of the output of the previous time step as an input to the current time step. This recursive mechanism creates dependencies across the sequence, allowing RNNs to process variable-length sequences and capture temporal relationships within the data.

The fundamental building block of RNNs are the RNN cells, which combine the current input with the previous hidden state to produce a new hidden state. At each time step, the input updates the cell’s hidden state, enabling the model to learn data patterns over time. This architecture allows the network to maintain memory and summarize relevant information from the sequence.

A simplified version of the structure of a RNN is shown in the figure below:

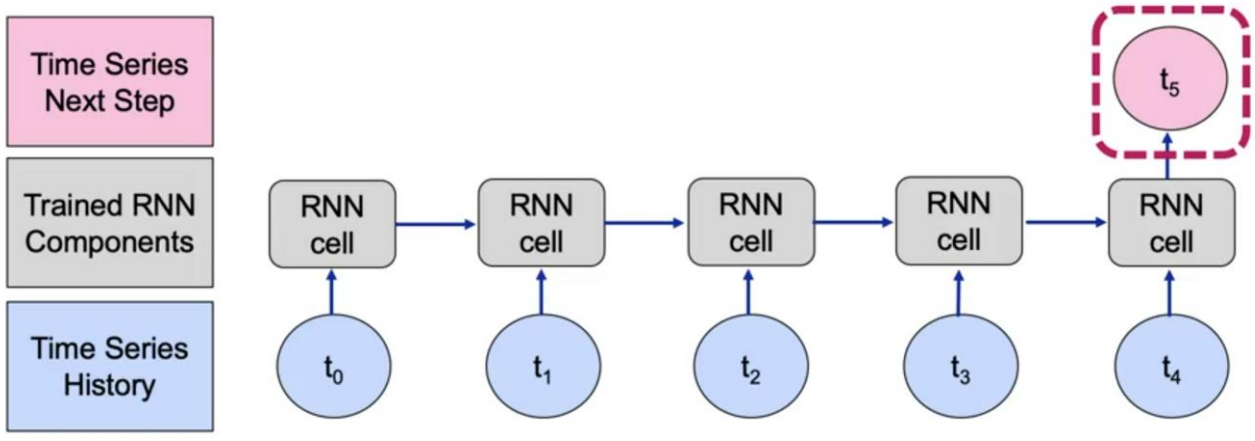


Figure 1: Structure of a RNN, from : “Specialized Models: Time Series and Survival Analysis”- IBM, Coursera

The process begins with input t_0, which is passed to the first hidden state H_0. Then, H_0 is combined with the next time step t_1,resulting in H_1, which now contains information from t_0, H_0 and t_1. Similarly H_2 will have information on t_0, H_0, t_1, H_1 and t_2. The process continues until the final hidden state, that will generate the prediction for the next time step.

To forecast beyond a single future step, the previously predicted value can be fed as input into the next time step.

The update of the hidden state is computed as:

$$H_i = \sigma\left(Ut_i + VH_{(i-1)}\right)$$

Where σ is the Sigmoid function, that outputs values in $[0,1]$:

$$\sigma(x) = \frac{e^x}{e^x + 1}$$

The output (i.e. the next step in the time series) is compute as:

$$t_{out} = WH_{(out-1)}$$

In these equations, U, V and W are trainable weight matrices that are applied on data as linear transformations, while the sigmoid function is used to introduce non-linearity in data. The values of the weights are learned through the backpropagation algorithm, that adjusts parameters in order to minimize a cost function.

Despite their ability to capture complex dynamics, RNNs struggle in modeling long term dependencies over many time steps.

2.4.2 Long Short Term Memory (LSTM) Networks

The Long Short Term Memory (LSTM) networks are a type of recurrent neural network specifically designed to address the limitations of RNNs when modeling long term dependencies in sequential data.

LSTMs introduce a memory cell that employes structures known as “gates”. There are three main gates:

- Input gate: controls which information enters the memory cell.
- Forget gate: determines what information to discard from the memory cell.
- Output gate: decides what information is passed on from the memory cell.

In this way, the network can select the piece of information to retain over extended sequences, allowing it to learn long-term dependencies. The image below illustrates this process:

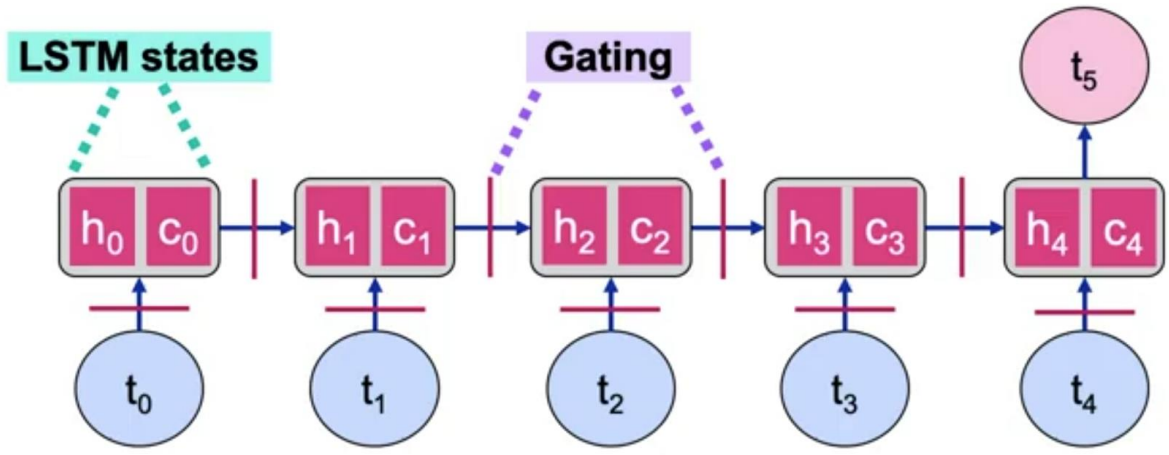


Figure 2: Structure of a LSTM network, from : “Specialized Models: Time Series and Survival Analysis”- IBM, Coursera

A variation of LSTM, known as Bidirectional LSTM , processes sequences both in forward and backward directions. This architecture consists of two LSTM layers whose outputs are then combined to form the final prediction. This approach enables the network to learn longer dependencies in sequential data, compared to traditional LSTMs.

Finally, while LSTM are effective in capturing long term dependencies and complex temporal patterns, they require a large number of trainable parameters, resulting in higher training times and increased the risk of overfitting.

2.4.3 Gated Recurrent Unit

The Gated Recurrent Unit (GRU) was introduced as a more efficient version of LSTM, aiming to reduce its computational cost while maintaining a similar performance. Although GRUs use the same architecture as LSTMs, they simplify the internal structure by merging its gates and eliminating the separate cell state, relying solely on the hidden state. This design reduces the number of parameters, making GRUs faster to train and reduces the computational complexity.

In terms of forecasting accuracy, LSTMs are often preferred for tasks requiring very long-term dependencies, while GRUs have been shown to perform comparably in many practical applications, especially when computational efficiency is crucial.

3. Case Study: Walmart weekly sales prediction

3.1 Introduction

The objective of this study is to compare and evaluate different time-series forecasting methods and assess their effectiveness in predicting weekly sales. Through this analysis, we aim to determine the most accurate approach for forecasting weekly sales and explore how external factors, such as economic conditions, can affect prediction accuracy. Finally, we seek to outline the opportunities and managerial implications that Walmart can leverage in optimizing its weekly sales forecasts.

This study is based on a dataset containing information from 45 Walmart stores across the US. The dataset contains details on the weekly sales for each store and department, as well as relevant information about the socio-economic conditions of the region. One of the main challenges of this analysis is that the available sales data covers only two years, which may limit the ability to capture long-term trends and seasonality. However, the dataset remains extensive, as it includes sales data for 45 stores and 90 departments, providing a solid basis for meaningful insights.

Our objective is to use this dataset to forecast future demand across the stores and departments, consequently, the target variable is the “Weekly Sales” variable.

3.2 Dataset

The dataset we used was retrieved from a past Kaggle competition hosted by Walmart. To build the final dataset, three datasets were merged:

1. Train: Contains data on weekly sales per store and department.
2. Features: Contains weekly data on economic factors (CPI and employment), temperature, fuel price, holidays.
3. Stores: Contains data on the size and type of stores.

The final dataset contains 421,570 observations and 16 columns, representing weekly sales data for 45 Walmart stores in a time period that goes from 2010-02-05 to 2012-10-26 (143 weeks). Each store is made up of 81 departments, from an excel file found online the name of departments were retrieved.

3.2.1 Univariate analysis

This section provides an overview of all the variables included in the dataset, summarizing key insights from descriptive statistics and visualization. This part is crucial to assess the overall structure of the data.

The Store variable is a categorical variable that assigns a unique identifier (from 1 to 45) to each of the stores. On average, each store has 9305 observations, with the store having the fewest observations recording 6151 entries. Overall, we can conclude that each store is adequately represented in the dataset.

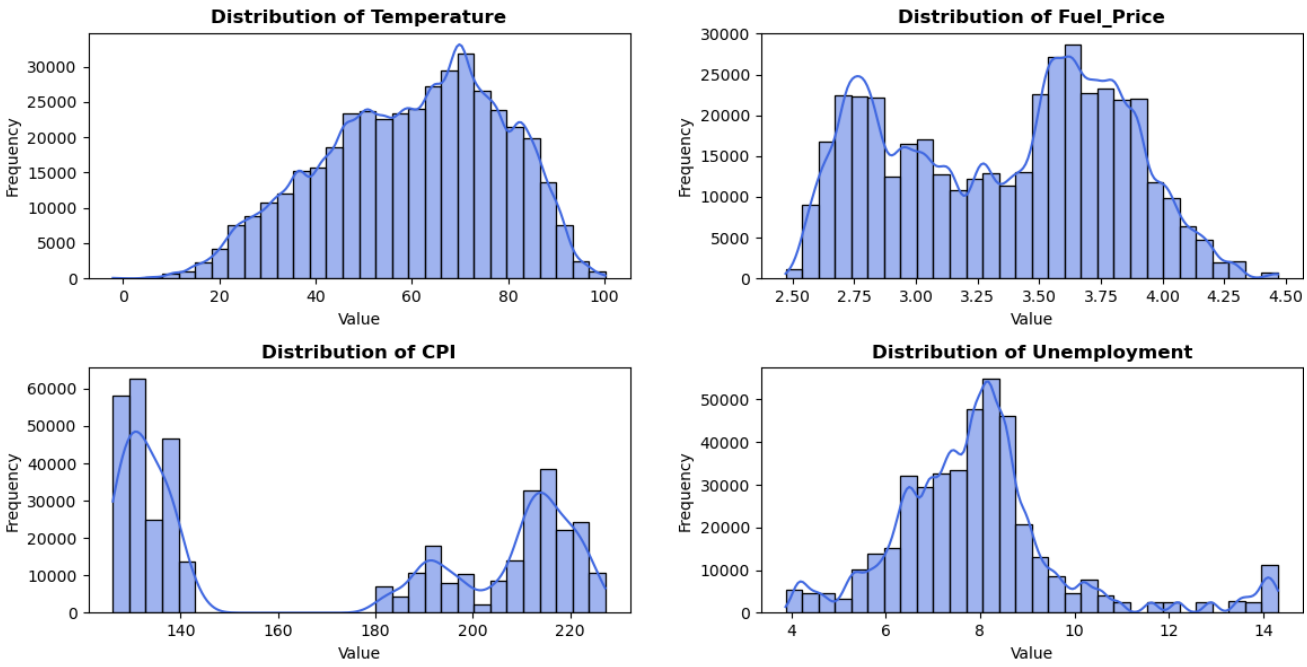
Stores are grouped by Type, there are three type of stores: A,B and C. Type A stores have the largest Size (average size = 182197.3), type B stores are of medium size (average = 101777.6) and type C stores are the smallest (average = 40536.1). More than half of the stores are type A stores, whereas only 10.1% of the stores are type C stores. However, the dataset does not provide explicit information on what the size variable measures, meaning we cannot determine whether it refers to metrics such as square footage or the number of employees.

Each store contains multiple Departments, with 81 unique departments and each department is identified by a number (from 1 to 99). To get a better understanding of each department, they were grouped them in seven broader Categories: Home & Lifestyle, Health & Wellness, Food & Beverage, Electronics & Media, Apparel & Accessories, Miscellaneous and Unknown department. Each category contains approximately 12 departments and departments are evenly distributed among categories.

The Date variable represents the temporal dimension of the dataset: each week is identified with the first date of that week. The dataset covers a 143 weeks from 2010-02-05 to 2012-10-26, with observations evenly distributed across years. To facilitate analysis, additional time-related variables— Year, Month and Week —were extracted. A Boolean variable IsHoliday, indicates whether a given corresponds to an Holiday. The holidays considered are: Thanksgiving, Christmas, Super bowl and Labor day and 7% of observations were recorded during holiday weeks.

Temperature, Fuel Price, CPI (Consumer Price Index) and Unemployment are regional and economic variables that may influence weekly sales. To get a better understanding of these variables the histograms were plotted:

- Temperature: the histogram exhibits a normal-like distribution, which is expected for temperature data.
- Fuel Price: fuel prices fluctuate within a few key pricing ranges and tend to stabilize around specific values, likely due to economic policies.
- CPI (Consumer Price Index): the histogram has a bimodal structure, probably indicating a shift in inflation or cost-of-living over time.
- Unemployment: the distribution is right-skewed, showing that most regions have moderate unemployment rates, while some experience significantly higher values.



Figures 3-6: Histogram of the distribution of Temperature, Fuel Price, CPI and Unemployment

The variables AMD1 and AMD2 (Log of Average Markdown 1 and Log of Average Markdown 2), represent anonymized data about promotional activities carried out by Walmart. Their distribution is characterized by distinct integer values, indicating these variables have undergone a log transformation. More insights about these variables will be given in the next paragraphs.

The target variable Weely_Sales, represents the total sales for a given store and department in a specific week. The histogram of Weekly_Sales shows a strongly right skewed distribution, meaning most stores have relatively low weekly sales, while a few outliers have extremely high sales.

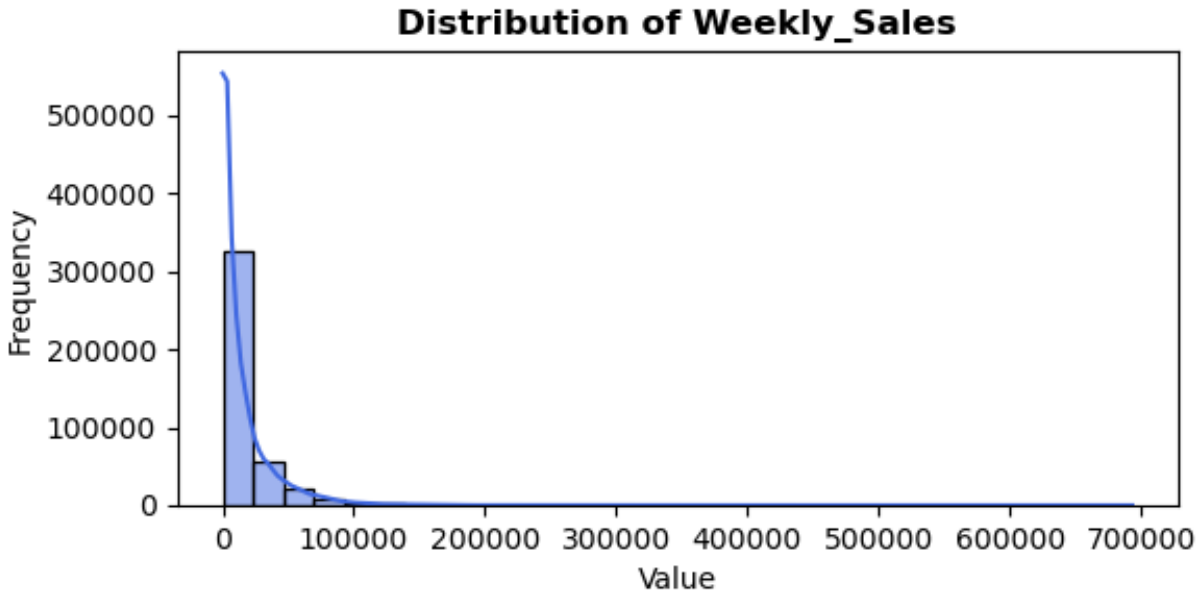


Figure 7: Histogram of the distribution of Weekly sales

3.3 Data preparation

For the purpose of this analysis, the dataset was already cleaned and ready to be used for modelling. The main steps of the data cleaning and preparation phase are reported below.

3.3.1 Handling missing and unexplained values

- Deletion of unexplained values: negative values for weekly sales were removed, as they do not hold practical significance.
- Missing values: the markdown variables (MD1 to MD5) exhibited a high percentage of missing values (66% on average). Consequently, they were predicted using a Random Forest model ($R^2 = 0.589$).

3.3.2 Handling outliers

- Weekly Sales: outliers in weekly sales were not removed since it is the target variable, and eliminating them could result in the loss of valuable information.
- Markdown Variables(1-5) : these variables contained a significant proportion of outliers. To handle this issue, we grouped markdowns into two categories:
    - AMD1: represents the average of markdowns peaking in February (MD1 and MD4).
    - AMD2: represents the average of markdowns peaking in December (MD2, MD3, and MD5).

This grouping approach was implemented because we lacked detailed information regarding the meaning of these markdown variables. Additionally, a logarithmic transformation was applied to address the skewness of these variables.

3.4.Preliminary analysis

Before delving into predictive modelling, it is essential to develop a comprehensive understanding of the dataset. This section aims to explore key relationships between variables and identify temporal patterns that may impact sales behavior.

3.4.1 Bivariate analysis

To examine the relationships between variables we conducted a bivariate analysis using a correlation matrix:

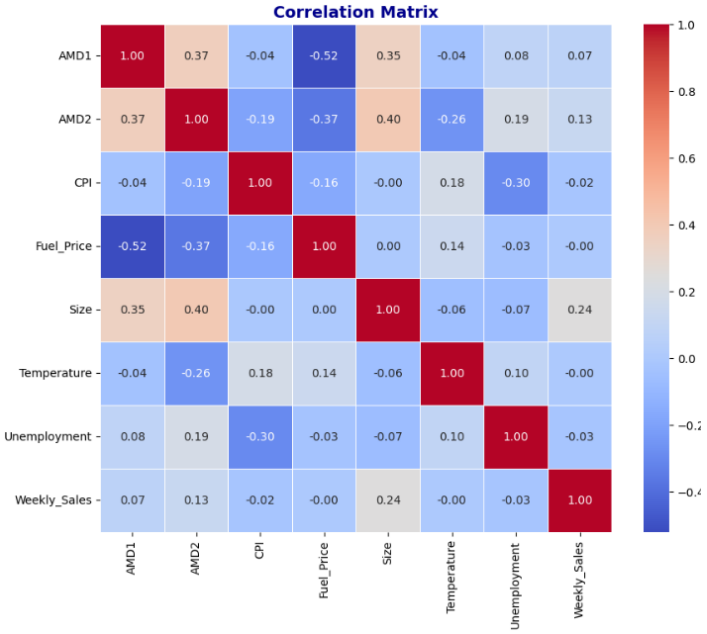


Figure 8: heatmap of the correlation between numerical variables

Below are reported the main findings from the analysis of the correlation matrix:

- Markdowns are correlated with size, in fact larger stores make promotions more frequently than smaller ones.
- Markdowns are negatively correlated with fuel price, a possible reason is that higher fuel prices reduce disposable income, leading to decreased spending on non-essentials products on which promotions are usually ran.
- Unemployment and CPI are negatively correlated. In fact, higher unemployment typically reduces consumers' spending, leading to lower demand-driven inflation.
- Weekly sales have a moderate positive correlation with Store Size (0.24), indicating that larger stores tend to have higher sales. Promotional activities (AMD1 and AMD2) have weak positive correlations with sales, suggesting a limited impact. Economic factors and temperature show negligible correlations, implying they do not directly influence sales in this dataset.

3.4.2 Time series analysis

In this section the target variable weekly sales is analyzed to understand its behavior over time. This step is essential as it provides insights into the structure of this variable, which is one of the key elements in determining the most appropriate forecasting method.

To achieve this, three graphs were plotted: (1) weekly sales variation over the entire period, (2) average sales across months, comparing three years, and (3) weekly sales trends, also comparing three years.

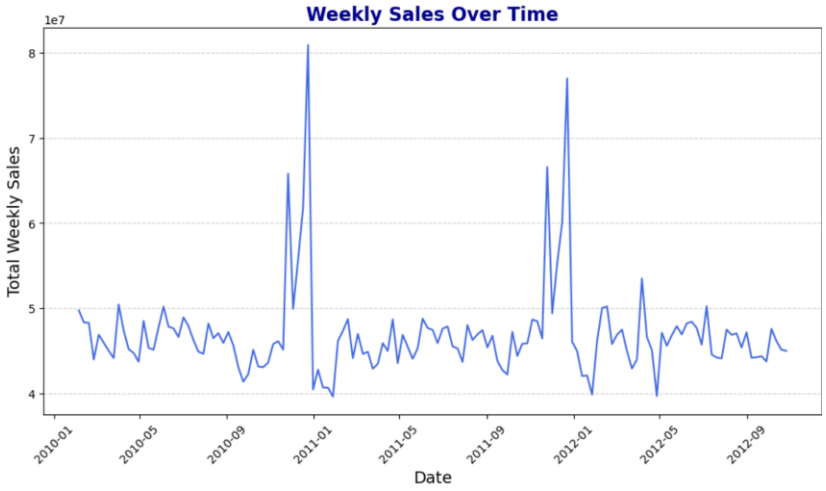


Figure 9: Weekly sales over time

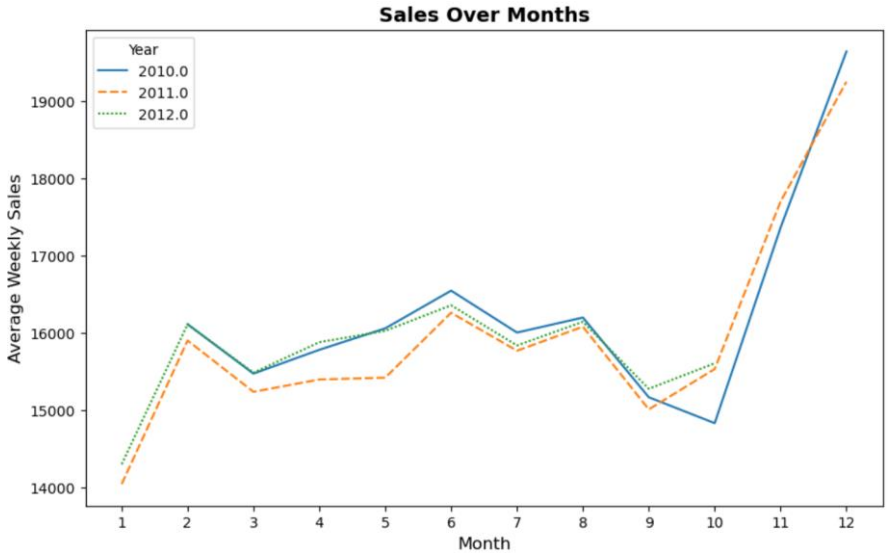


Figure 10: Weekly sales by months, comparing years

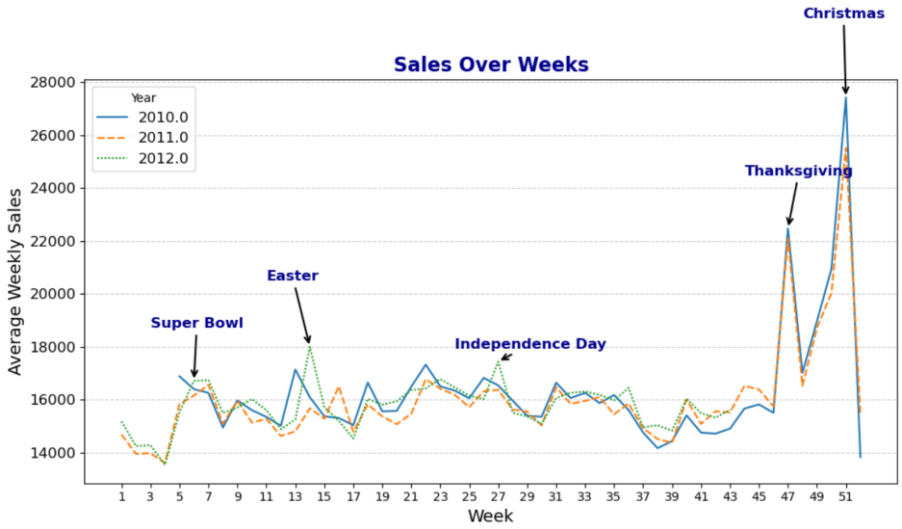


Figure 11: Weekly sales by week of the year, comparing years and highlighting holidays

The findings from these visualizations indicate that, overall, sales remain relatively stable from January to October, with a sharp peak from November to December, particularly during holiday weeks. A clear seasonal pattern emerges, with significant spikes occurring at regular intervals(holiday periods). However, a formal statistical analysis is required to confirm these seasonal trends. From the previous analysis, the time series does not seem to be stationary, to validate this hypothesis the ACF plot was analyzed.

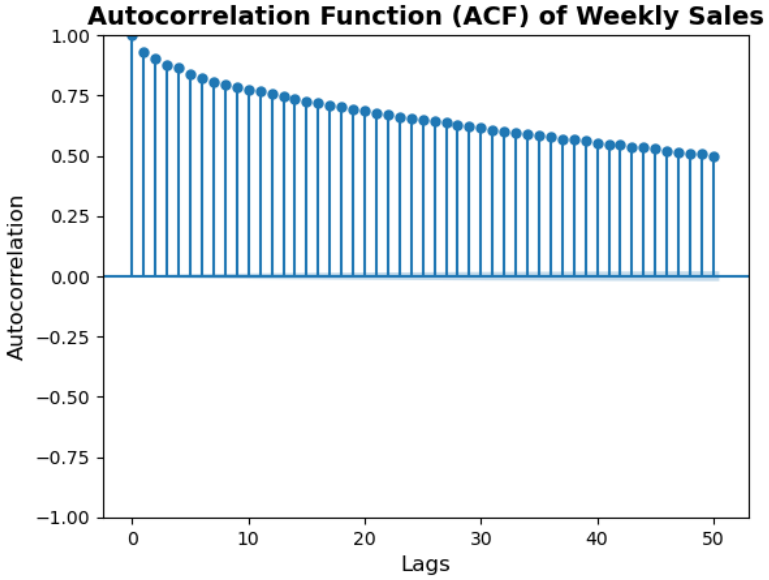


Figure 12: ACF plot of weekly sales

The Autocorrelation Function (ACF) plot shows slow decay over time which is characteristic of non-stationary time series, where past values influence future values. Additionally, the skewness of the distribution of weekly sales, is a further indicator of non-stationarity as it indicates that the mean is not constant over time.

However, the Augmented dickey-fuller test, a widely used method to detect stationarity, fails to reject the null hypothesis of stationarity. This result may be due to the limited time span of the dataset (less than three years), which provides only about two full seasonal cycles, potentially insufficient for detecting seasonality. Despite the ADF test result, the overall characteristics of the data suggest that the series is non-stationary.

To further analyze the structure of weekly sales, an STL decomposition was used:

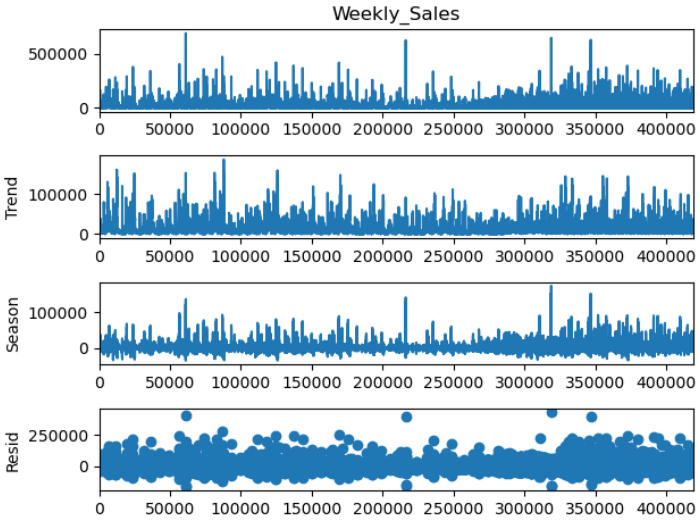


Figure 13: STL decomposition of weekly sales

The trend component of the decomposition fluctuates significantly over time, indicating that there is no clear long-term upward or downward trajectory in the data. The seasonality component shows repeating fluctuations, although their magnitude is not perfectly constant, indicating that seasonal effects are present but may vary in intensity over different time periods. Overall, the decomposition and the run-length plot confirms that the data exhibits a seasonal pattern, making it essential to consider seasonality in further modeling and forecasting efforts.
To better understand the data's seasonality, STL decomposition was applied to each category. The analysis showed clear seasonal patterns in *Food & Beverage* and *Health & Wellness*. In contrast, *Electronics & Media* showed irregular patterns likely driven. Other categories showed moderate or weak seasonality.

3.5 Forecasting methods

This section explores different methods to forecast weekly sales, aiming to identify the most suitable approach. The methods applied are: exponential smoothing (single, double, triple), ARIMA, SARIMAX, Prophet and GRU.

One of the main challenges risen in this phase is the limited temporal coverage of the dataset. Despite having over 400,000 observations, the data is segmented by stores and further divided into departments, which complicates modeling. Additionally, the dataset spans less than two full cycles, potentially hindering the detection of seasonality in certain forecasting models.

To evaluate the effectiveness of each method the Mean Absolute Error (MAE), Root Mean Squared Error (RMSE) are compared:

- The Mean Absolute Error (MAE) is computed as:

$$MAE = \frac{1}{n}\sum_{t=1}^{n}|y_t - \hat{y}_t|$$

It represents the average of the absolute differences between actual and predicted values, as a result, it describes how much the predictions deviate from the actual values. This measure is relatively easy to interpret and is not sensitive to outliers, however it does not account for the scale of data

- The Root Mean Square Error (RMSE) is computed as:

$$RMSE = \sqrt{\frac{1}{n}\sum_{t=1}^{n}(y_t - \hat{y_t})^2}$$

With this measure, larger errors have a bigger impact on the final value; however, it is less straightforward to interpret due to squaring

While the Mean Absolute Percentage Error (MAPE) is commonly used for evaluating time series models, it was excluded from this analysis because the distribution of Weekly_Sales includes many small and near-to-zero values. Since MAPE expresses errors as percentages of the actual values, small denominators can inflate the percentage error dramatically, leading to distorted and unreliable results.

3.5.1 Exponential Smoothing

The first method applied is exponential smoothing. Due to its simplicity and computational efficiency, this method will serve as the baseline method. Single, double and triple exponential smoothing have been applied. The obtained results are summarized in the table below:

	MAE	RMSE
Single Exponential Smoothing	2619.2	3190.4
Double Exponential Smoothing	4114.7	4854.8
Triple Exponential Smoothing	2257.9	2822.0

Form the analysis of the error metrics, we notice that across all models, the RMSE is significantly larger than the MAE, indicating the presence of outliers in some store-department combinations: in fact, RMSE penalizes large errors more heavily, making it more sensitive to outliers and large deviations in predictions. Furthermore, for Single Exponential Smoothing and Triple Exponential Smoothing, the MAE distribution is heavily right-skewed, meaning that the majority of store-department combinations have a relatively low MAE. In contrast, Double Exponential smoothing exhibits a spread of MAE errors, and the higher MAE and RMSE indicate that adjusting for trends may have led to overfitting or introduced unnecessary complexity, especially when data did not have any strong trend pattern.

The comparison of the errors metrics obtained by the three models shows that Triple Exponential Smoothing outperforms both Single and Double Exponential Smoothing (DES) in terms of forecast accuracy. While Single Exponential Smoothing provides a reasonable baseline, its inability to capture trends or seasonality limits its performance. Double Exponential Smoothing performs worse, probably due to the fact that many

store-department couples do not exhibit a strong trend. Finally, Triple Exponential Smoothing achieves the best results, indicating that seasonality plays a crucial role in the data.

To ensure sufficient data for model estimation, Single and Double Exponential Smoothing were performed on store-department couples having at least 20 observations, while Triple Exponential Smoothing was furtherly restricted on combinations that had at least two full seasonal cycles in the data. This differentiation may have led to a distortion in the comparative results, potentially biasing the performance metrics in favor of triple smoothing due to higher data quantity in the filtered sample.

3.5.2 ARIMA family forecasting methods

As a next step, two models from the ARIMA family were applied: ARIMA and SARIMAX.

A key challenge of this phase was tuning the model's parameters. For ARIMA, the auto_arima function was used to compute the parameters of each store-department combination. This widely used tool automates the selection of optimal parameters by performing a stepwise search across different combinations of (p, d, q) to minimize a criterion such as AIC or BIC, where AIC and BIC are statistical measures that evaluate the model quality by balancing goodness of fit and model complexity. Although alternative tuning methods exist (such as manual selection based on domain knowledge or grid search) auto_arima was chosen for its balance between automation and accuracy. This approach provided an efficient solution for ARIMA, given its relatively low computational demands.

However, applying the same approach for SARIMAX significantly increased computational time, due to the complexity introduced by exogeneous variables and seasonality. To address this issue, auto_arima was performed on 20 combinations of store-departments and the obtained parameters were then averaged and applied across all the combinations in the dataset. In this way a reasonable balance between parameter reliability and computational efficiency was achieved.

As before, to ensure sufficient data for model estimation, ARIMA and SARIMAX were performed on store-department couples having at least 20 observations.

The performance metrics of the two models are reported in the table below:

	MAE	RMSE
ARIMA	2321.2	2847.2
SARIMAX	2001.9	2567.8

The analysis of error distribution reveals that SARIMAX exhibits a more concentrated distribution of both MAE and RMSE, with the majority of store-department combinations clustered around low error values.

The comparison of error metrics clearly indicated that SARIMAX significantly outperforms ARIMA. The better performance of SARIMAX can be attributed to its ability to incorporate exogenous variables and its explicit modeling of seasonality. In fact, SARIMAX adjusts forecasts more accurately in the presence of cyclical fluctuations that are not captured by ARIMA. Additionally, the inclusion of external regressors allows the model to factor in variables that may influence sales, such as promotions or holidays, further improving forecasting accuracy.

3.5.3 Prophet

Subsequently, after the issues encountered in computing parameters for the models in the ARIMA family, Facebook's Prophet was performed on the dataset.

	MAE	RMSE
Prophet (no regressors)	1711.2	2314.5
Prophet scaled regressors	2364.7	3081.5
Prophet with scaled size	1710.7	2313.8

In order to improve the performance of Prophet, regressors were introduced to test how much adding external variables could improve the forecasting accuracy. Regressors were scaled in order to ensure that variables with different magnitudes did not distort the model's training process. However, this model performed significantly worse than the version without regressors. This underperformance might be due to the fact that the majority of regressors have low levels of correlation with the target variable, with correlation coefficients generally below 0.3, and the inclusion of weak predictors may have introduced noise rather than improving the signal.

Subsequently, only the "Size" variable was used as a regressor in the model, due to its relatively high correlation with Weekly_Sales. This adjustment slightly improved the model's performance, however, the performance metrics are almost identical, indicating that although the variable "Size" contributes some predictive power, it is not a strong enough driver of weekly sales to make a significant difference.

Overall, Prophet demonstrates ease of use, particularly in tuning parameters and handling seasonality or special events automatically. This built-in flexibility makes it well-suited for forecasting tasks involving time series with regular seasonal patterns, like weekly sales.

3.5.4 Recurrent Neural Networks – GRU

Finally, recurrent neural networks were employed to forecast weekly sales, with Gated Recurrent Unit networks chosen for their ability to capture temporal dependencies and complex patterns in the data, as well as their computational efficiency, as GRU is generally faster than LSTMs models.

Tuning the hyperparameters of the GRU model represented a key challenge. Several key parameters, such as the number of hidden units, the number of layers, the dropout

rate, the sequence length, the learning rate, and the batch size, had to be carefully selected to balance model complexity and generalization performance. This process involved iterative testing, supported by early stopping and learning rate scheduling, to avoid overfitting while ensuring convergence. Given the high dimensionality of the data and the variability across store-department combinations, finding a configuration that worked well across all scenarios required extensive experimentation and fine-tuning.

Three GRU architectures were tested: GRU with no additional input, GRU with lagged values of weekly sales (specifically, sales from the previous one and two weeks) and a bidirectional GRU with the same lagged features. The performance metrics are presented in the table below:

	MAE	RMSE
GRU (no lags)	4341.9	8340.1
GRU with lag_1, lag_2	1539.14	3097.38
Bidirectional GRU with lag_1, lag_2	1598.3	3110.6

The results of the GRU-based models highlight the importance of incorporating lagged features in time series forecasting. When no lags are included, GRU struggles to capture the temporal structure of the data, leading to limited predictive performance. This suggests that short-term historical patterns are highly informative for forecasting weekly sales, which is consistent with the nature of retail time series where recent trends and seasonality strongly influence future values.

The bidirectional GRU performs slightly worse than the unidirectional GRU, which may be attributed to the increased model complexity introduced by processing sequences in both directions. This added complexity can lead to overfitting that worsens the performance metrics. As a matter of fact, a more complex architecture may capture noise or spurious temporal correlations that do not hold in unseen data, ultimately reducing the model's predictive performance. This finding indicates that more complex architectures do not necessarily translate to better results, especially in datasets with high variability and noise.

Despite the improvements in mean absolute error, the root mean squared error remains relatively high. This discrepancy suggests that while the GRU model performs well on average, it struggles to accurately forecast instances with extreme deviations from the norm. These outliers or sudden shifts in the data result in large individual errors, which disproportionately inflate the RMSE due to its sensitivity to large residuals, indicating that GRU may have limited ability to capture abrupt changes or rare events in the sales patterns.

3.6 Conclusions

3.6.1 Model comparison

The aim of this section is to compare the results of the different models in order to identify the most suitable approach for forecasting Walmart's weekly sales, while also highlighting the strengths and limitations of each model.

Among the exponential smoothing models, Triple Exponential Smoothing clearly outperforms the Single and Double variants, emphasizing the importance of modelling seasonality in retail time series. A stronger performance is observed with the ARIMA family models, particularly SARIMAX, which is able to incorporate both seasonal components and exogenous variables. However, it remains difficult to isolate the exact source of this improvement—whether it stems more from the inclusion of seasonal patterns or from the use of external regressors.

Interestingly, in Facebook's Prophet the introduction of multiple regressors actually worsened the model performance, likely because many of the external variables used had only weak correlations with the target. This suggests that in the absence of strong and meaningful relations between regressors and the target variable, the introduction of regressors may lead to overfitting.

On the deep learning front, the GRU model with lagged sales inputs achieved the lowest MAE among all models, reflecting its ability to capture time series dependencies and complex patterns. However, the model also recorded a relatively high RMSE, revealing a struggle to generalize in presence of outliers or sudden changes, which are common in retail environments due to the presence of promotions or holidays.

Taking into account predictive accuracy, implementation complexity and computational efficiency, Prophet emerges as the most balanced and pragmatic model. Prophet provides the best RMSE and, although it does not achieve the lowest MAE, its MAE is close to GRU's and requires significantly lower computational demands and far easier parameter tuning. Furthermore, Prophet's automatic treatment of holidays and special events (which do not necessarily follow standard seasonal patterns) proves to be especially advantageous in retail data. As a matter of fact, visual inspection(figure 11) of weekly sales patterns reveals consistent spikes aligned with major holidays and festivities, which Prophet is programmed to model automatically. This likely explains its improved RMSE performance over more GRU, which may not capture such non-periodic but recurring effects unless explicitly encoded.

Therefore, while GRU holds potential for a more accurate modeling with further tuning and data enrichment, Prophet currently offers the best trade-off between performance and usability, making it particularly suitable for retail forecasting tasks where domain-specific effects like holidays play a major role.

3.6.2 Applications

Forecasting weekly sales on a store-department level, can provide Walmart with relevant insights to support data-driven decisions across different areas of its operations and value chain.

One of the most straightforward application is inventory management. Forecasts allow for a dynamic adjustment of supply deliveries, helping stores being prepared during peak sales weeks and reducing overstock and wastes during weeks with less sales.

Additionally, staffing and workforce planning may be improved through forecasting: for example, the number of employes should be increased during holiday periods and the need for seasonal hires can be evaluated in advance based on the expected sales volume

Furthermore, forecasting improves data-driven budgeting and revenue forecasting: instead of relying solely on historical yearly trends, Walmart may align its revenue projections and budgets with week-by-week expectation.

Finally, forecasting is particularly valuable for benchmarking: comparing predicted and actual sales, Walmart can identify underperforming stores or departments, and define a strategy for these elements. Additionally, forecasted sales provide a valuable baseline to evaluate the effectiveness of marketing campaigns or new product launches, helping assess whether such initiatives truly drive additional value.

3.6.3 Limitations and areas for improvement

One of the greatest challenges encountered during this analysis was the limited availability of complete data, particularly the lack of full seasonal cycles for many store-department combinations. In several cases, time series were too short to effectively model seasonality. Additionally, the low context and availability of promotional variables (AMD1 and AMD2) significantly constrained the ability to capture the effects of promotions on sales. These features, if complete and well-integrated, could have substantially improved the model's accuracy, especially in a retail context where promotions play a critical role in driving demand.

Another limitation was the restricted use of external variables, as only a small set of regressors was available and many lacked strong correlation with the target. Incorporating more informative external factors (such as local events, marketing spend, or competitor activity) could significantly enhance the models' ability to capture demand drivers and improve forecasting accuracy.

To address these limitations, future work should aim to extend the scope of the analysis by including a larger time window, thereby ensuring more full seasonal cycles across all store-department pairs and collect missing values and data from key variables that might influence weekly sales.

4. Conclusions

This study presented an exploration of various time series forecasting methods, with a particular focus on approaches commonly used to predict sales in a retail environment. By combining both a theoretical and practical approach through the Walmart's case study, it evaluated a range of different approaches: from classical statistical models to modern frameworks like Prophet and deep learning models.

A key finding is the importance of understanding the structure of a time series before selecting a forecasting method. Analyzing elements such as trend, seasonality and cycles - using diagnostic tools such as the ACF plot and the STL decomposition - is crucial in the model selection and parameter tuning. In the Walmart case, these tools revealed the non-stationarity of the data, as well as its seasonality.

Simpler models like moving averages and exponential smoothing perform adequately on stationary time series, but may struggle in presence of significant variability. Among them, triple exponential smoothing was particularly effective in modelling the seasonality of data, although it struggled when full seasonal cycles were not present in the data. SARIMAX offered improved accuracy by capturing both autocorrelation and seasonal patterns, and by integrating exogenous variables. However, the model's success heavily depends on the correct specification of parameters, which can be time-consuming and easily lead to mistakes.

Prophet emerged as a powerful and user-friendly alternative, striking a balance between automation and accuracy. Its built-in ability to handle holidays and changepoints made it particularly suitable for business applications like retail forecasting, in fact, the model achieved strong results in the case study, even without extensive parameter tuning.

Deep learning approaches, particularly GRUs, represent a new frontier for time series forecasting. When provided with lagged sales values as input, GRU performed reasonably well in the case study, however it still had difficulties in handling outliers or sudden changes in data. Furthermore, bidirectional GRU did not improve performance, highlighting that increasing model complexity is not always beneficial.

Overall, while deep learning models offer great potential, they require large volumes of training data and time series with multiple seasonal cycles, and may require significant time. In contrast, models like Prophet demonstrate how automation and flexibility can deliver competitive results with far lower complexity, making them ideal in many business contexts.

Ultimately, this study reaffirms that there is no universally superior forecasting model. The optimal forecasting method depends on data availability, characteristics of the time series and computational constraints.

Time series forecasting has broad applicability across industries and functions—from sales and inventory management to budgeting, energy demand, stock market predictions, and supply chain optimization.

In conclusion, by combining theoretical insights, data-driven tools, and scalable techniques, businesses can unlock the value hidden in time series data and turn into actionable insights that can truly drive companies' competitive advantage.

References

1. *Brockwell, P. J., & Davis, R. A. (2016). Introduction to Time Series and Forecasting (3rd ed.). Springer.*
2. *Business Research Insights. (2025). Time Series Forecasting Market Size, Share, Growth, and Industry Analysis, By Type (Software, Service), By Application (Business Planning, Financial Industry, Medical), Regional Insights and Forecast To 2033. https://www.businessresearchinsights.com/market-reports/time-series-forecasting-market-114943*
3. *KPMG. (2016). Forecasting with Confidence. https://assets.kpmg.com/content/dam/kpmg/pdf/2016/07/forecasting-with-confidence.pdf*
4. *Coursera. Specialized Models: Time Series and Survival Analysis. IBM. https://www.coursera.org/learn/time-series-survival-analysis*
5. *Hyndman, R. J., & Athanasopoulos, G. (2021). Forecasting: Principles and Practice (3rd ed.) – ACF- ARIMA- Prophet. https://otexts.com/fpp3/*
6. *Kis, A. N. (2024, July 26). Understanding Autocorrelation and Partial \Autocorrelation Functions (ACF and PACF). Medium. https://medium.com/@kis.andras.nandor/understanding-autocorrelation-and-partial-autocorrelation-functions-acf-and-pacf-2998e7e1bcb5*
7. *Nau, R. Moving Average and Exponential Smoothing Models. Duke University. https://people.duke.edu/~rnau/411avg.htm*
8. *GeeksforGeeks. (2024). Exponential Smoothing for Time Series Forecasting. https://www.geeksforgeeks.org/exponential-smoothing-for-time-series-forecasting/*
9. *GeeksforGeeks. (2025). SARIMA. https://www.geeksforgeeks.org/sarima-seasonal-autoregressive-integrated-moving-average/*
10. *DataScientest. SARIMAX Model: What is it? How can it be applied to time series? https://datascientest.com/en/sarimax-model-what-is-it-how-can-it-be-applied-to-time-series*
11. *Facebook. Prophet Forecasting Tool. https://facebook.github.io/prophet/*
12. *Google Cloud. What is Deep Learning? https://cloud.google.com/discover/what-is-deep-learning*

13. *IEEE. (2024). Deep Learning-Based Hybrid Forecasting Model for Multistep Time Series Prediction. https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=10583885*
14. *GeeksforGeeks. Introduction to Long Short-Term Memory (LSTM). https://www.geeksforgeeks.org/deep-learning-introduction-to-long-short-term-memory/*
15. *GeeksforGeeks. Gated Recurrent Unit Networks. https://www.geeksforgeeks.org/gated-recurrent-unit-networks/*
16. *Kaggle. Walmart Recruiting - Store Sales Forecasting. https://www.kaggle.com/competitions/walmart-recruiting-store-sales-forecasting/overview*
17. *Medium. Understanding Evaluation Metrics for Time Series Forecasting. https://eshban9492.medium.com/understanding-evaluation-metrics-for-time-series-forecasting-5c8a3c877654*